



UNIVERSIDADE VIRTUAL DO ESTADO DE SÃO PAULO

Curso de Ciências de Dados e Engenharia da Computação

Disciplina: Projeto Integrador I

SISTEMA DE CONTROLE OPERACIONAL CONTÁBIL

Guia de Implantação com Docker Compose

Integrantes:

Caio Henrique Ferreira Cavalcanti

Emmanuele de Moura Peres

Lucas Magalhães Nunes

Renato da Silva Cunha

Vinicius Fernandes dos Santos

Wagner Silva Matutino

Tutora: Adriana Costa de Souza

SISTEMA DE CONTROLE OPERACIONAL CONTÁBIL

Guia de Implantação com Docker Compose

Documentação técnica apresentada na disciplina de Projeto Integrador I para o curso de Ciências de Dados e Engenharia da Computação da Universidade Virtual do Estado de São Paulo (UNIVESP).

Tutora: Adriana Costa de Souza

Sumário

1	PRÉ-REQUISITOS	1
2	ESTRUTURA DOS SERVIÇOS	1
3	SUBINDO A APLICAÇÃO	1
3.1	Clonar ou baixar o projeto	1
3.2	Configurar variáveis de ambiente (opcional)	1
3.3	Iniciar os containers	2
3.4	Aguardar a inicialização	2
4	ACESSANDO O SISTEMA	2
5	UTILIZANDO A APLICAÇÃO	3
5.1	Login no sistema	3
5.2	API REST via Swagger	3
6	PARANDO A APLICAÇÃO	3
7	COMANDOS ÚTEIS	3
7.1	Executar comandos Django dentro do container	4
8	SOLUÇÃO DE PROBLEMAS COMUNS	4
8.1	Porta já em uso	4
8.2	Banco de dados corrompido ou dados inconsistentes	4
8.3	Permissão negada no entrypoint	4

1 PRÉ-REQUISITOS

Para executar o sistema via Docker, é necessário ter instalado:

- **Docker** (versão 20 ou superior): <https://docs.docker.com/get-docker/>
- **Docker Compose** (versão 2 ou superior): incluído no Docker Desktop ou instalável separadamente

Para verificar se as ferramentas estão instaladas, execute no terminal:

```
docker --version
docker compose version
```

2 ESTRUTURA DOS SERVIÇOS

O arquivo `docker-compose.yml` define dois serviços que sobem juntos:

Serviço	Porta local	Descrição
backend	8000	API Django (REST + Swagger)
frontend	3000	Interface web React (servida via Nginx)

3 SUBINDO A APLICAÇÃO

3.1 Clonar ou baixar o projeto

```
git clone https://github.com/CaioSix/ProjetoIntegradorI.git
cd ProjetoIntegradorI
```

3.2 Configurar variáveis de ambiente (opcional)

Copie o arquivo de exemplo e edite conforme necessário:

```
cp .env.example .env
```

As variáveis disponíveis são:

Variável	Descrição
DJANGO_SECRET_KEY	Chave secreta do Django — alterar em produção
DJANGO_DEBUG	Modo debug: True ou False
DJANGO_ALLOWED_HOSTS	Hosts permitidos (padrão: *)
DATABASE_PATH	Caminho do banco SQLite dentro do container
DJANGO_SUPERUSER_USERNAME	Usuário administrador criado automaticamente
DJANGO_SUPERUSER_PASSWORD	Senha do administrador

Importante: para uso em produção, altere obrigatoriamente `DJANGO_SECRET_KEY`, defina `DJANGO_DEBUG=False` e altere `DJANGO_SUPERUSER_PASSWORD`.

3.3 Iniciar os containers

```
docker compose up --build
```

O argumento `--build` reconstrói as imagens. Nas execuções seguintes, se não houver alterações no código, pode-se omiti-lo:

```
docker compose up
```

Para executar em segundo plano (*modo detached*):

```
docker compose up -d
```

3.4 Aguardar a inicialização

Na primeira execução, o backend executa automaticamente:

- Aplicação das migrações do banco de dados;
- Carregamento dos dados iniciais (*fixtures*);
- Criação do superusuário administrador.

As mensagens exibidas no terminal serão:

```
contabil_backend | ==> Aplicando migracoes...
contabil_backend | ==> Carregando dados iniciais...
contabil_backend | ==> Criando superusuario...
contabil_backend | ==> Iniciando servidor em 0.0.0.0:8000...
```

4 ACESSANDO O SISTEMA

Após os containers subirem, os seguintes endereços estarão disponíveis:

Endereço	Descrição
http://localhost:3000	Interface web (frontend React)
http://localhost:8000/api/docs/	Documentação interativa da API (Swagger)
http://localhost:8000/admin/	Painel administrativo do Django

Credenciais padrão: usuário admin / senha admin123

5 UTILIZANDO A APLICAÇÃO

5.1 Login no sistema

1. Acesse `http://localhost:3000`;
2. Informe o usuário e senha (`admin / admin123`);
3. Após autenticar, o painel de controle será exibido com as seguintes seções:
 - **Dashboard** — visão geral das empresas e tarefas;
 - **Tarefas** — controle de tarefas contábeis por competência, com filtros por status, empresa e período;
 - **Empresas** — cadastro e gestão de empresas clientes;
 - **Obrigações** — obrigações fiscais vinculadas a tipos de empresa.

5.2 API REST via Swagger

1. Acesse `http://localhost:8000/api/docs/`;
2. Clique em **Authorize** e informe as credenciais para obter o token de acesso;
3. Utilize o token nos endpoints protegidos preenchendo Token `<seu-token>` no campo de autorização;
4. Explore e teste todos os endpoints disponíveis diretamente na interface.

6 PARANDO A APLICAÇÃO

Para parar os containers preservando os dados:

```
docker compose down
```

Para parar e remover o volume do banco (apaga todos os dados):

```
docker compose down -v
```

7 COMANDOS ÚTEIS

Comando	Descrição
<code>docker compose logs -f</code>	Acompanha os logs em tempo real
<code>docker compose logs backend</code>	Logs apenas do backend
<code>docker compose ps</code>	Lista os containers em execução
<code>docker compose restart backend</code>	Reinicia apenas o backend
<code>docker compose exec backend bash</code>	Abre terminal dentro do container

7.1 Executar comandos Django dentro do container

Criar novo superusuário:

```
docker compose exec backend python manage.py createsuperuser
```

Abrir o shell do Django:

```
docker compose exec backend python manage.py shell
```

Aplicar migrações manualmente:

```
docker compose exec backend python manage.py migrate
```

8 SOLUÇÃO DE PROBLEMAS COMUNS

8.1 Porta já em uso

```
Error: Bind for 0.0.0.0:8000 failed: port is already allocated
```

Solução: encerre o processo que usa a porta ou altere o mapeamento no `docker-compose.yml` (ex.: "8080:8000").

8.2 Banco de dados corrompido ou dados inconsistentes

```
docker compose down -v  
docker compose up --build
```

8.3 Permissão negada no entrypoint

```
chmod +x entrypoint.sh  
docker compose up --build
```